



Discussion 10

Distributed Systems

04/19/24

Staff

Announcements

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
					HW5 Checkpoint Deadline	
		HW 5 Due		Midterm 3		
				Project 3 Due		

Transactions

Transactions

Transactions are indivisible units.

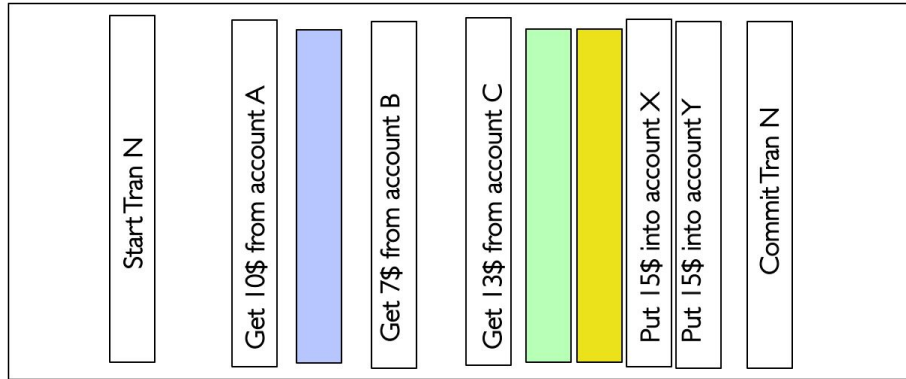
Transactions follow **ACID properties**.

- **Atomicity:** a transaction must occur in its entirety or not at all.
- **Consistency:** a transaction takes system from one consistent state (i.e. meets all integrity and correctness constraints) to another.
- **Isolation:** each transaction must *appear* to execute on its own.
- **Durability:** a committed transaction's changes must persist through crashes.

Each entry in a transaction must be **idempotent**, meaning it has the same effect when executed once or many times) (i.e. $f(f(x)) = f(x)$).

Transactions are stored on **logs/journals** (on disk).

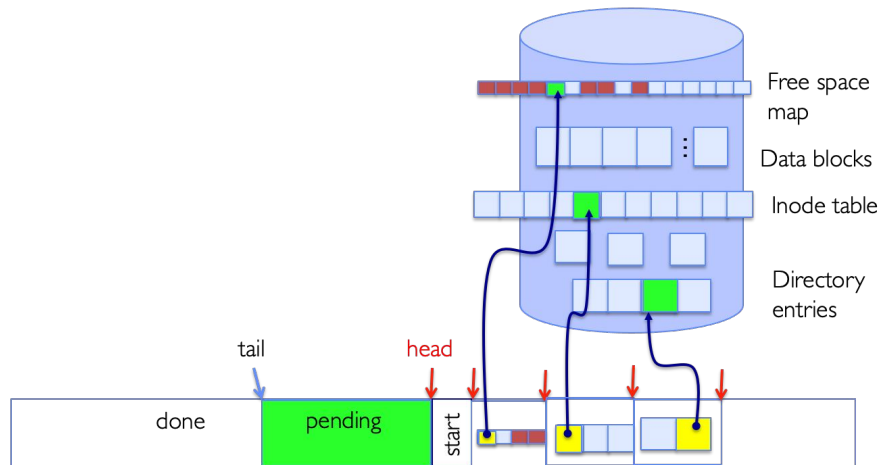
- Writes to a log are assumed to be atomic.
- Logs will typically use a circular buffer (maintain head, tail pointer).



Transactions

Transactional file systems use transactions for reliability.

- **Journaling file systems** use **write-ahead logging (WAL)** where log entries are written to disk *before* data gets modified.
 - Preparation phase append all updates to the log.
 - Commit phase logs a commit record, indicating the transaction must happen at some point.
 - Write back will *asynchronously* apply committed transactions' updates.
 - Garbage collect any completed transaction in the background.
- **Log structured file system (LFS)** use logs as storage.



Log: in non-volatile storage (Flash or on Disk)

Concept Check

1. Why does each entry in a transaction need to be idempotent?
2. Consider a file system with a buffer cache. Your program creates a file called `pintos . bean`. While the changes have been logged with a commit entry, the tail of the log still points to before the start of the log entry corresponding to creating `pintos . bean`. Assuming no further disk reads or writes have happened, if your program wants to read `pintos.bean`, will it need to scan through the logs?
3. What is the purpose of a commit entry in a log?

Concept Check

1. Why does each entry in a transaction need to be idempotent?

Entry might already have been applied prior to crash.

2. Consider a file system with a buffer cache. Your program creates a file called `pintos . bean`. While the changes have been logged with a commit entry, the tail of the log still points to before the start of the log entry corresponding to creating `pintos . bean`. Assuming no further disk reads or writes have happened, if your program wants to read `pintos.bean`, will it need to scan through the logs?
3. What is the purpose of a commit entry in a log?

Concept Check

1. Why does each entry in a transaction need to be idempotent?

Entry might already have been applied prior to crash.

2. Consider a file system with a buffer cache. Your program creates a file called `pintos . bean`. While the changes have been logged with a commit entry, the tail of the log still points to before the start of the log entry corresponding to creating `pintos . bean`. Assuming no further disk reads or writes have happened, if your program wants to read `pintos.bean`, will it need to scan through the logs?

No since buffer cache will hold blocks corresponding to `pintos.bean`.

3. What is the purpose of a commit entry in a log?

Concept Check

1. Why does each entry in a transaction need to be idempotent?

Entry might already have been applied prior to crash.

2. Consider a file system with a buffer cache. Your program creates a file called `pintos . bean`. While the changes have been logged with a commit entry, the tail of the log still points to before the start of the log entry corresponding to creating `pintos . bean`. Assuming no further disk reads or writes have happened, if your program wants to read `pintos.bean`, will it need to scan through the logs?

No since buffer cache will hold blocks corresponding to `pintos.bean`.

3. What is the purpose of a commit entry in a log?

Makes each transaction atomic.

Journaling

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies.

You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.

1. What are the possible states of files F1 and F2 on disk at boot time?
 - a. F1 has nothing applied.
 - b. F1 is in an intermediate state.
 - c. F1 has all its updates written to disk.
 - d. F2 has nothing.
 - e. F2 is in an intermediate state.
 - f. F2 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies.

You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.

1. What are the possible states of files F1 and F2 on disk at boot time?
 - a. F1 has nothing applied.
 - b. F1 is in an intermediate state.
 - c. F1 has all its updates written to disk.
 - d. F2 has nothing.
 - e. F2 is in an intermediate state.
 - f. F2 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies. You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.

2. Say the following entries are found at the end of the log.

7. Add a directory.
8. COMMIT

How does this change the possible states of file F2 on disk at boot time?

- a. F2 has nothing.
- b. F2 is in an intermediate state.
- c. F2 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies. You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.

2. Say the following entries are found at the end of the log.

7. Add a directory.
8. COMMIT

How does this change the possible states of file F2 on disk at boot time?

- a. F2 has nothing.
- b. F2 is in an intermediate state.
- c. F2 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies. You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.
7. [T2] Add a directory.
8. [T2] COMMIT

3. Say the log contained only entries (5) through (8) as shown. What are the possible states of file F1 on disk at the time of the reboot?
 - a. F1 has nothing applied.
 - b. F1 is in an intermediate state.
 - c. F1 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies. You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.
7. [T2] Add a directory.
8. [T2] COMMIT

3. Say the log contained only entries (5) through (8) as shown. What are the possible states of file F1 on disk at the time of the reboot?
 - a. F1 has nothing applied.
 - b. F1 is in an intermediate state.
 - c. F1 has all its updates written to disk.

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies.

You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.
7. [T2] Add a directory.
8. [T2] COMMIT

4. When recovering from a system crash and applying the updates recorded in the journal, does the OS need to check if these updates were partially applied before the failure?

Journaling

You create two new files, F1 and F2, right before your laptop's battery dies.

You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. [T1] Find free blocks, mark free map accordingly.
2. [T1] Allocate inode.
3. [T1] Add directory entry.
4. [T1] COMMIT
5. [T2] Find free blocks, mark free map accordingly.
6. [T2] Allocate inode.
7. [T2] Add a directory.
8. [T2] COMMIT

4. When recovering from a system crash and applying the updates recorded in the journal, does the OS need to check if these updates were partially applied before the failure?

No. Each log entry is idempotent.

Distributed Systems

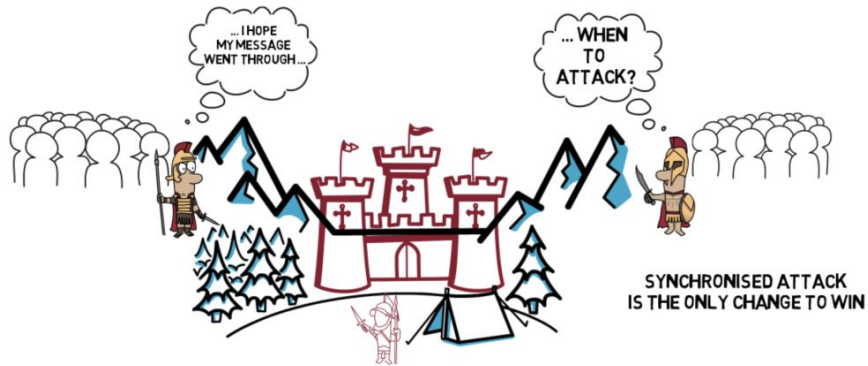
Distributed Transactions

General's paradox demonstrates the impossibility of reaching consensus over an *unreliable* network.

- Two generals on separate mountains who can only communicate via messengers that may be captured.
- Need to attack at the same time for attack to succeed.

Distributed transaction involves multiple machines *atomically* agreeing to do something or not.

- No constraints on time like the General's paradox → easier problem and can be solved.

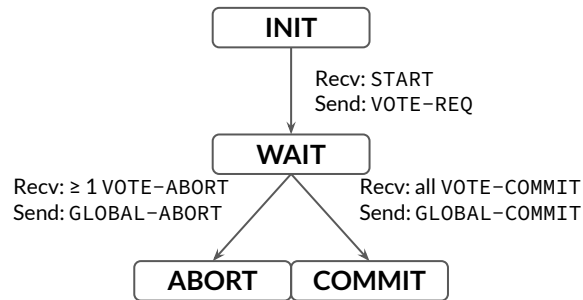


Atomic Commit

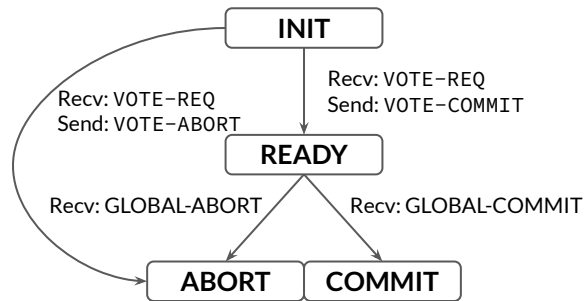
Two-phase commit (2PC) achieves distributed transaction as an **atomic commit protocol (ACP)**.

- Developed by Jim Gray (Berkeley Professor, Turing Award winner)!
- Assumes one coordinator, N participants, and a persistent log for each machine.
- Must come to a unanimous decision (i.e. commit requires all N workers to vote commit).
- **Prepare phase**
 1. Coordinator sends **VOTE-REQ** to all participants.
 2. Each participant decides to commit/abort and logs **VOTE-COMMIT/VOTE-ABORT**.
 3. Each participant sends **VOTE-COMMIT/VOTE-ABORT** to the coordinator.
 4. Coordinator logs **GLOBAL-COMMIT/GLOBAL-ABORT** based on participants' decisions.
- **Commit/abort phase**
 1. Coordinator sends **GLOBAL-COMMIT/GLOBAL-ABORT** to all participants.
 2. Each participant logs **GLOBAL-COMMIT/GLOBAL-ABORT** and commits/aborts.
 3. Each participant sends an **ACK** to the coordinator.
 4. Coordinator ends transaction once all N **ACKs** have been received.
- **Single node (participant or coordinator) failure leads to blocking.**

Coordinator State Machine



Participant State Machine



Concept Check

1. The vanilla implementation of 2PC logs all decisions. How could 2PC be optimized to reduce logging?
2. 2PC exhibits blocking behavior where a worker can be stalled until the coordinator recovers. Why is this undesirable?

Concept Check

1. The vanilla implementation of 2PC logs all decisions. How could 2PC be optimized to reduce logging?
[Abort decisions can be ignored with the idea of presumed abort.](#)
2. 2PC exhibits blocking behavior where a worker can be stalled until the coordinator recovers. Why is this undesirable?

Concept Check

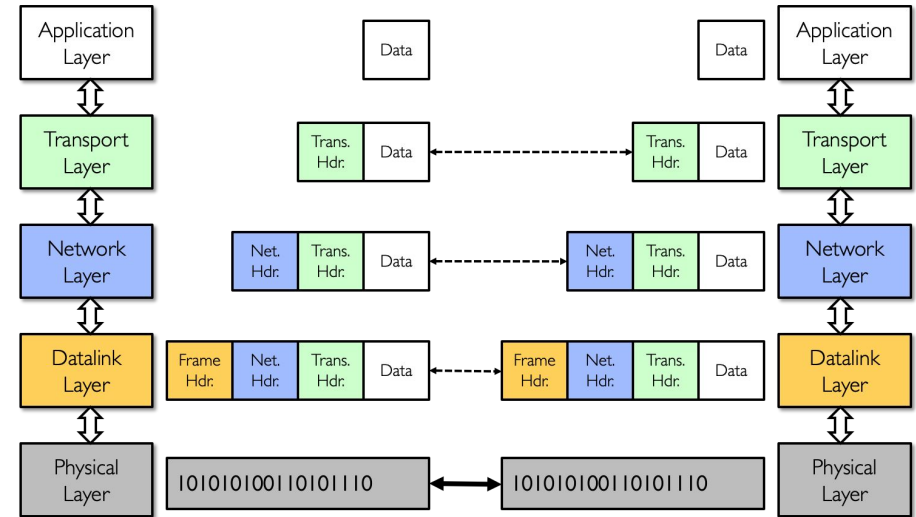
1. The vanilla implementation of 2PC logs all decisions. How could 2PC be optimized to reduce logging?
[Abort decisions can be ignored with the idea of presumed abort.](#)
2. 2PC exhibits blocking behavior where a worker can be stalled until the coordinator recovers. Why is this undesirable?
[Worker blocked → may be holding resources that other transactions may need.](#)

Networking

Network is a physical connection that allows two computers to communicate.

Layering means building complex services from simpler ones where each layer provides services needed by higher layers by utilizing services provided by lower layers.

- **Physical layer** delivers the raw bits from endpoint to another (e.g. ethernet, fiber)
- **Datalink layer** adds the packet abstraction and is responsible for local delivery of packets within the same network (e.g. switches, bridges, network interface cards).
- **Network layer** is responsible for global delivery of packets across one or more networks.
- **Transport layer** provides end to end communication, operating on streams of data (e.g. TCP, UDP).
- **Application layer** provides network support for applications (e.g. HTTP, FTP, DNS, SSH).

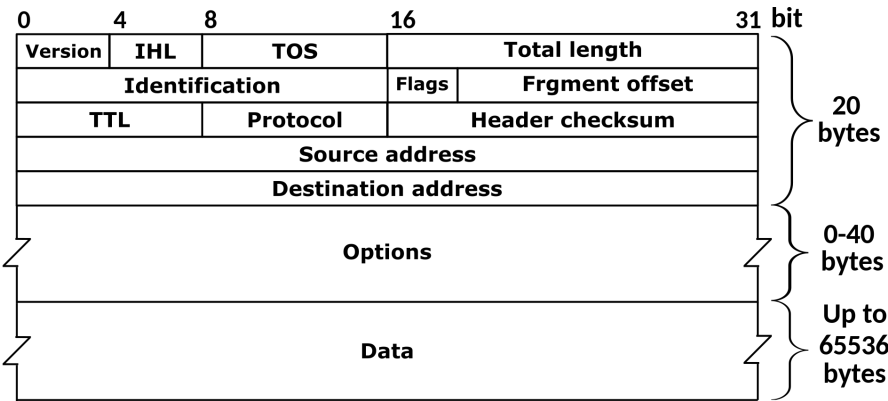


Networking

Network layer serves as the “narrow waist” to facilitate interoperability between applications and physical mediums.

Internet protocol (IP) facilitates the network layer as a “best-effort” packet delivery system.

- Packets may be lost, corrupted, or delivered out of order.
- **IP Address** is a 32-bit (IPv4) or 64-bit (IPv6) destination of packet (e.g. 192.168.162.162).



Networking

Achieved routing from machine to machine in network layer, but want routing from process to process (using ports).

User datagram protocol (UDP) provides best effort delivery.

- No-frills extension of IP other than a **checksum** to detect some forms of packet corruption.
- Low-overhead due to small packet size!

Transmission control protocol (TCP) provides reliable, in-order delivery.

- Error-checked delivery, flow control, congestion control.
- Much bigger overhead than UDP.

Both of these protocols are built on top of IP in the transport layer!



"Hi, I'd like to hear a TCP joke."

"Hello, would you like to hear a TCP joke?"

"Yes, I'd like to hear a TCP joke."

"OK, I'll tell you a TCP joke."

"Ok, I will hear a TCP joke."

"Are you ready to hear a TCP joke?"

"Yes, I am ready to hear a TCP joke."

"Ok, I am about to send the TCP joke. It will last 10 seconds, it has two characters, it does not have a setting, it ends with a punchline."

"Ok, I am ready to get your TCP joke that will last 10 seconds, has two characters, does not have an explicit setting, and ends with a punchline."

"I'm sorry, your connection has timed out. Hello, would you like to hear a TCP joke?"

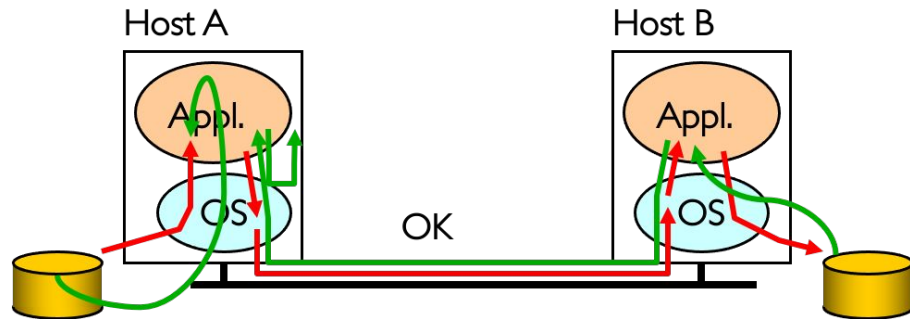
End-to-End Principle

“End-to-End Arguments in System Design” by Saltzer, Reed, Clark (1984)

- Hugely influential paper, serves as basis for a lot of systems.

End-to-end (E2E) principle is the idea that some types of network functionality can only be correctly implemented end-to-end.

- Conservative interpretation: don't implement a function at lower levels unless it can be completely implemented at this level.
- Moderate interpretation: if hosts can implement functionality correctly, implement it in a lower level only as a performance enhancement.
- Many different interpretations!



Concept Check

3. An interpretation of the E2E Principle argues that functionality should only be placed in the network if certain conditions are met.

Only If Sufficient: Don't implement a function in the network unless it can be completely implemented at this level.

Only If Necessary: Don't implement anything in the network that can be implemented correctly by the hosts.

Only If Useful: If hosts can implement functionality correctly, implement it in the network only as a performance enhancement.

Consider the example of the reliable packet transfer: making all efforts to ensure that a packet sent is not lost or corrupted and is indeed received by the other end. Using each of the three criteria, argue if reliability should be implemented in the network.

4. Why would you ever want to use UDP over TCP?

Concept Check

3. An interpretation of the E2E Principle argues that functionality should only be placed in the network if certain conditions are met.

Only If Sufficient: Don't implement a function in the network unless it can be completely implemented at this level.

Only If Necessary: Don't implement anything in the network that can be implemented correctly by the hosts.

Only If Useful: If hosts can implement functionality correctly, implement it in the network only as a performance enhancement.

Consider the example of the reliable packet transfer: making all efforts to ensure that a packet sent is not lost or corrupted and is indeed received by the other end. Using each of the three criteria, argue if reliability should be implemented in the network.

Only If Sufficient: No. End hosts still need to check.

Only If Necessary: No. Reliability can be implemented fully in the end hosts.

Only If Useful: Sometimes. Lossy links may lead to severe performance decrease with only an E2E check.

4. Why would you ever want to use UDP over TCP?

Concept Check

3. An interpretation of the E2E Principle argues that functionality should only be placed in the network if certain conditions are met.

Only If Sufficient: Don't implement a function in the network unless it can be completely implemented at this level.

Only If Necessary: Don't implement anything in the network that can be implemented correctly by the hosts.

Only If Useful: If hosts can implement functionality correctly, implement it in the network only as a performance enhancement.

Consider the example of the reliable packet transfer: making all efforts to ensure that a packet sent is not lost or corrupted and is indeed received by the other end. Using each of the three criteria, argue if reliability should be implemented in the network.

Only If Sufficient: No. End hosts still need to check.

Only If Necessary: No. Reliability can be implemented fully in the end hosts.

Only If Useful: Sometimes. Lossy links may lead to severe performance decrease with only an E2E check.

4. Why would you ever want to use UDP over TCP?

UDP used in applications that prioritize speed and low overhead, don't care about being lossy (e.g. streaming AV, real time data).

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1 , W_2 , W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

1. What is the minimum amount of time needed for 2PC to complete successfully?

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

1. What is the minimum amount of time needed for 2PC to complete successfully?

Prepare phase: worker receives message + log result + send back message + coordinator logs.

$$\max(400 + 10 + 400, 300 + 20 + 300, 200 + 30 + 200) + 5 = 815 \text{ ms}$$

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

1. What is the minimum amount of time needed for 2PC to complete successfully?

Prepare phase: worker receives message + log result + send back message + coordinator logs.

$$\max(400 + 10 + 400, 300 + 20 + 300, 200 + 30 + 200) + 5 = 815 \text{ ms}$$

Commit/abort phase: worker receives message + log result + send back message (no coordinator log)

$$\max(400 + 10 + 400, 300 + 20 + 300, 200 + 30 + 200) = 810 \text{ ms}$$

$$\text{Total latency} = 810 + 815 = 1625 \text{ ms}$$

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

Transaction still commits since a commit decision was made.

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

Transaction still commits since a commit decision was made.

Preparation phase: 815 ms as before

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

Transaction still commits since a commit decision was made.

Preparation phase: 815 ms as before

Commit phase:

First try takes 3000 ms (timeout)

Second try only takes $300 + 20 + 300 = 620$ ms (W_2)

Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1, W_2, W_3).

The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

Transaction still commits since a commit decision was made.

Preparation phase: 815 ms as before

Commit phase:

First try takes 3000 ms (timeout)

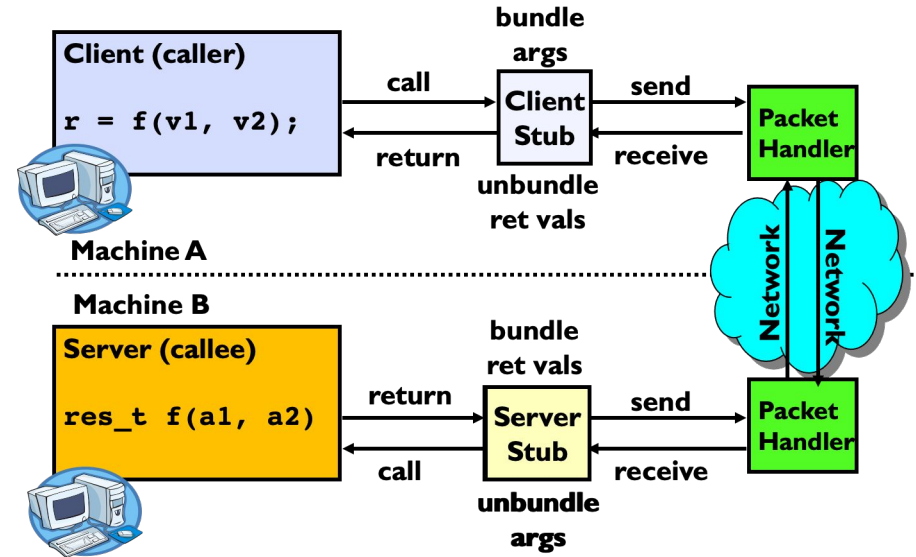
Second try only takes $300 + 20 + 300 = 620$ ms (W_2)

Total latency = $815 + 3000 + 620 = 4435$ ms

Remote Procedure Call (RPC)

Remote procedure call (RPC) makes executing code on a remote machine as simple and straightforward as calling a local function.

- Automate all of the complexity of translating between representations.
- **Marshalling/serializing** involves converting values into a canonical form (e.g. endianness), serializing objects, copying arguments passed by reference, etc.
- **Stub** provides the glue on client/server.
 - **Client stub** is responsible for marshalling/serializing arguments and unmarshalling/deserializing return values.
 - **Server stub** is responsible for unmarshalling/deserializing arguments and marshalling/serializing return values.



Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

```
/* Assume dest has enough space allocated. */
void read_args(int sock_fd, char *dest) {
    int byte_len = 0;
    int read_bytes = 0;
    while ((read_bytes = read(sock_fd, dest, 8 - byte_len)) > 0) {
        byte_len += read_bytes;
    }
}
```

1. Edward notices that the arguments required are either 4 or 8 bytes. As a result, he believes he can handle either case by attempting to read 8 bytes with the code on the left. However when Edward implements it, he notices that for some inputs the server appears to be stuck. Why might this be happening and for which inputs could this happen?

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

```
/* Assume dest has enough space allocated. */
void read_args(int sock_fd, char *dest) {
    int byte_len = 0;
    int read_bytes = 0;
    while ((read_bytes = read(sock_fd, dest, 8 - byte_len)) > 0) {
        byte_len += read_bytes;
    }
}
```

1. Edward notices that the arguments required are either 4 or 8 bytes. As a result, he believes he can handle either case by attempting to read 8 bytes with the code on the left. However when Edward implements it, he notices that for some inputs the server appears to be stuck. Why might this be happening and for which inputs could this happen?

read is trying to return 8 bytes from the socket or indicate there is no more data. However, the socket will not return 0 unless the socket is closed. As long as the connection stays open, the read will block instead.

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);

/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while (_____) {
        bytes_read += cur_read;
    }
    id = _____;

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while (_____) {
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while (_____) {
        bytes_written += cur_written;
    }

    /* Clean up socket. */
    _____;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while ((cur_read = read(sock_fd, ((char *) &id) + bytes_read,
                           sizeof(uint32_t) - bytes_read)) > 0) {
        bytes_read += cur_read;
    }
    id = _____;

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while (_____) {
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while (_____) {
        bytes_written += cur_written;
    }

    /* Clean up socket. */
    _____;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while ((cur_read = read(sock_fd, ((char *) &id) + bytes_read,
                           sizeof(uint32_t) - bytes_read)) > 0) {
        bytes_read += cur_read;
    }
    id = ntohl(id);

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while (bytes_read < arg_bytes) {
        cur_read = read(sock_fd, args + bytes_read,
                        arg_bytes - bytes_read);
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while (bytes_written < ret_bytes) {
        cur_written = write(sock_fd, rets + bytes_written,
                           ret_bytes - bytes_written);
        bytes_written += cur_written;
    }

    /* Clean up socket. */
    close(sock_fd);
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while ((cur_read = read(sock_fd, ((char *) &id) + bytes_read,
                           sizeof(uint32_t) - bytes_read)) > 0) {
        bytes_read += cur_read;
    }
    id = ntohl(id);

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while ((cur_read = read(sock_fd, &args[bytes_read],
                           arg_bytes - bytes_read)) > 0) {
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while (1) {
        bytes_written += cur_written;
    }

    /* Clean up socket. */
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while ((cur_read = read(sock_fd, ((char *) &id) + bytes_read,
                           sizeof(uint32_t) - bytes_read)) > 0) {
        bytes_read += cur_read;
    }
    id = ntohl(id);

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while ((cur_read = read(sock_fd, &args[bytes_read],
                           arg_bytes - bytes_read)) > 0) {
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while ((cur_written = write(sock_fd, &rets[bytes_written],
                               ret_bytes - bytes_written)) > 0) {
        bytes_written += cur_written;
    }
    /* Clean up socket. */
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

2. Edward realizes his previous solution was insufficient, so he decides to implement a slightly more complicated protocol.

The client will perform the following.

1. Send an identifier for the function it wants as an integer (0 for `ith_prime`, 1 for `is_coprime`).
2. Send all bytes for all the arguments.

The server will then perform the following.

1. Read identifier.
2. Use identifier to allocate memory and set read size.
3. Read arguments.

```
void receive_rpc (int sock_fd) {
    /* Read in procedure identifier. */
    uint32_t id;
    int bytes_read = 0, cur_read = 0;
    while ((cur_read = read(sock_fd, ((char *) &id) + bytes_read,
                           sizeof(uint32_t) - bytes_read)) > 0) {
        bytes_read += cur_read;
    }
    id = ntohl(id);

    /* Get sizes and allocate space for arguments and return values. */
    char *args, *rets;
    size_t arg_bytes, ret_bytes;
    get_sizes(id, &args, &arg_bytes, &rets, &ret_bytes);

    /* Read in arguments. */
    bytes_read = 0;
    while ((cur_read = read(sock_fd, &args[bytes_read],
                           arg_bytes - bytes_read)) > 0) {
        bytes_read += cur_read;
    }

    /* Call appropriate server stub function based on id. */
    call_server_stub(id, args, arg_bytes, rets, ret_bytes);

    /* Write return values. */
    int bytes_written = 0, cur_written = 0;
    while ((cur_written = write(sock_fd, &rets[bytes_written],
                               ret_bytes - bytes_written)) > 0) {
        bytes_written += cur_written;
    }
    /* Clean up socket. */
    close(sock_fd);
}
```


Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    -----;
    -----;
    -----;
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(arg_ptr[0]);
    -----;
    -----;
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(arg_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    -----;
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(arg_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    memcpy(rets, &result, sizeof(uint32_t));
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    -----;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(arg_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    memcpy(rets, &result, sizeof(uint32_t));
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    -----;
    -----;
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(args_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    memcpy(rets, &result, sizeof(uint32_t));
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t x = ntohl(args_ptr[0]);
    uint32_t y = ntohl(args_ptr[1]);
    -----;
    -----;
}
```

Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(args_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    memcpy(rets, &result, sizeof(uint32_t));
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t x = ntohl(args_ptr[0]);
    uint32_t y = ntohl(args_ptr[1]);
    uint32_t result = htonl(is_coprime(x, y));
    -----;
}
```


Primes

Edward wants to build a RPC system for the following two procedures.

```
/* Returns the ith prime number (0-indexed). */
uint32_t ith_prime(uint32_t i);
```

```
/* Returns 1 if x and y are coprime, 0 otherwise. */
uint32_t is_coprime(uint32_t x, uint32_t y);
```

In particular, Edward is looking to setup the server side. You may assume the actual logic for both procedures have already been implemented.

3. Finally, Edward want to implement the call stubs, which are function wrappers between each individual function we support and the generic RPC library.

```
uint32_t ith_prime_cstub(struct addrinfo *addrs, uint32_t i) {
    uint32_t prime_val;
    i = htonl(i);
    call_rpc(addrs, RPC_PRIME, (char *)&i, 1, (char *) &prime_val, 1);
    return ntohl(prime_val);
}
```

```
uint32_t is_coprime_cstub(struct addrinfo *addrs, uint32_t x,
                          uint32_t y) {
    uint32_t coprime_val;
    uint32_t args[2] = {htonl(x), htonl(y)};
    call_rpc(addrs, RPC_COPRIME, (char *) args, 2,
              (char *) &coprime_val, 1);
    return ntohl(coprime_val);
}
```

```
void ith_prime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t i = ntohl(args_ptr[0]);
    uint32_t result = htonl(ith_prime(i));
    memcpy(rets, &result, sizeof(uint32_t));
}
```

```
void is_coprime_sstub (char *args, char *rets) {
    uint32_t* arg_ptr = (uint32_t *) args;
    uint32_t x = ntohl(args_ptr[0]);
    uint32_t y = ntohl(args_ptr[1]);
    uint32_t result = htonl(is_coprime(x, y));
    memcpy(rets, &result, sizeof(uint32_t));
}
```